

Question:

To exhibit different string operations

1.CharAt

Execution Steps:

Step 1: Start the program.

Step 2: The main() method begins execution.

Step 3: Declare and initialize the string.

```
String str = "Hello";
```

Step 4: Execute str.charAt(1).

Step 5: The character at index 1 is 'e'.

Step 6: Display the character using

```
System.out.println().
```

Step 7: Output is displayed as:

```
e
```

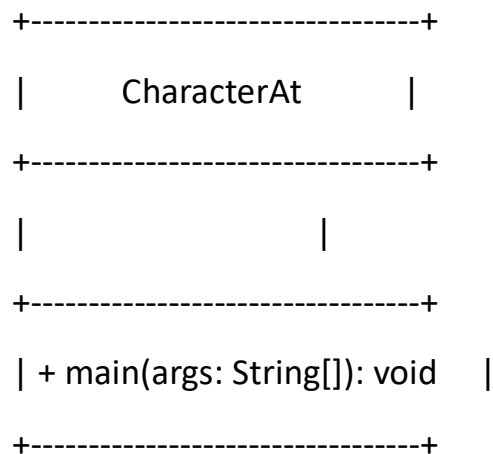
Step 8: Stop the program.

Algorithm:

1. Start.

2. Declare a class named CharacterAt.
3. Create the main() method.
4. Declare a String variable str and initialize it with the value "Hello".
5. Access the character at index 1 using charAt().
6. Display the character using System.out.println().
7. Stop.

UML Class Diagram:



Code:

```
package javacore;
```

```
public class CharacterAt {
```

```
public static void main(String[] args) {  
  
    String str = "Hello";  
  
    System.out.println(str.charAt(1));  
}  
}
```

Output:

e

2.StringComparison

Execution Steps:

Step 1: Start the program.

Step 2: The main() method begins execution.

Step 3: Initialize str1 with "Apple".

Step 4: Initialize str2 with "Banana".

Step 5: Execute str1.compareTo(str2).

Step 6: Compare the first characters:

'A' = 65

'B' = 66

Step 7: Calculate the difference:

$65 - 66 = -1$

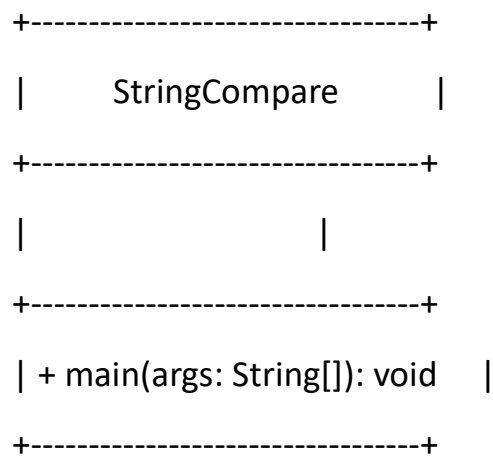
Step 8: Display the result using `println()`.

Step 9: Stop the program.

Algorithm:

1. Start.
2. Declare a class named `StringCompare`.
3. Create the `main()` method.
4. Declare two `String` variables `str1` and `str2`.
5. Initialize `str1` with "Apple" and `str2` with "Banana".
6. Compare the two strings using `compareTo()`.
7. Display the result.
8. Stop.

UML Class Diagram:



Output:

-1

3.StringConcat

Execution Steps:

Step 1: Start the program.

Step 2: The `main()` method begins execution.

Step 3: Declare and initialize `str1` with "Hello".

Step 4: Declare and initialize `str2` with "World".

Step 5: Execute `str1.concat(str2)`.

Step 6: The `concat()` method joins the two strings.

"Hello" + "World" = "HelloWorld"

Step 7: Display the concatenated string using

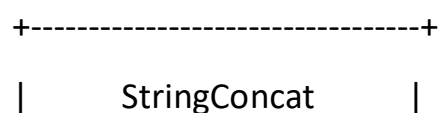
`System.out.println()`.

Step 8: Stop the program.

Algorithm:

1. Start.
2. Declare a class named StringConcat.
3. Create the main() method.
4. Declare two String variables str1 and str2.
5. Initialize str1 with "Hello" and str2 with "World".
6. Concatenate the two strings using concat().
7. Display the concatenated string.
8. Stop.

UML Class Diagram:



```

+-----+
|               |
+-----+
| + main(args: String[]): void    |
+-----+

```

Output:

HelloWorld

4.StringCreation

Execution Steps:

Step 1: Start the program.

Step 2: Execute the main() method.

Step 3: Initialize str with "Hello World".

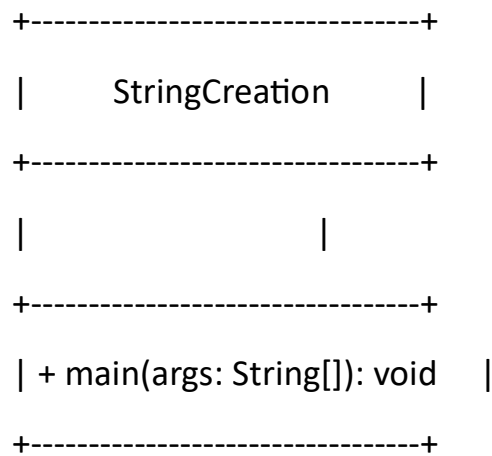
Step 4: Print the string using println().

Step 5: Stop the program.

Algorithm:

1. Start.
2. Declare a class StringCreation.
3. Create the main() method.
4. Initialize a String variable str with "Hello World".
5. Display str.
6. Stop.

UML Class Diagram:



Output:

Hello World

5.StringEndsWith

Execution Steps:

Step 1: Start the program.

Step 2: Execute the main() method.

Step 3: Initialize str with "Hello Java".

Step 4: Check whether the string ends with "Java".

Step 5: The condition is true.

Step 6: Display the result.

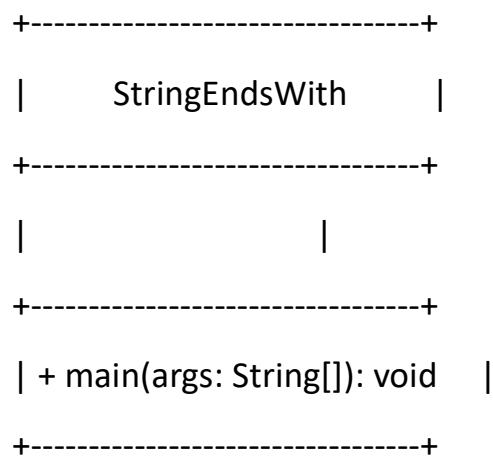
Step 7: Stop the program.

Algorithm:

1. Start.
2. Declare a class StringEndsWith.
3. Create the main() method.

4. Initialize str with "Hello Java".
5. Check whether str ends with "Java" using endsWith().
6. Display the result.
7. Stop.

UML Class Diagram:



Output:

true

6.StringEquals

Execution Steps:

Step 1: Start the program.

Step 2: Create str1 = "Hello".

Step 3: Create str2 = "Hello".

Step 4: Compare both strings using equals().

Step 5: Both strings contain the same text.

So, the result is true.

Step 6: Print true.

Step 7: Stop the program.

Algorithm:

START

Declare str1 = "Hello"

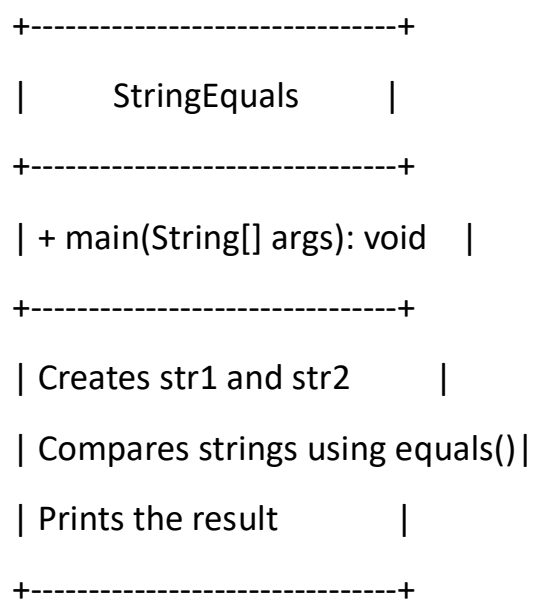
Declare str2 = "Hello"

Compare str1 and str2 using equals()

Print the result

STOP

UML Class Diagram:



Output:

true

7.StringEqualsIgnoreCase

Execution Steps:

Step 1: Start the program.

Step 2: Create str1 = "Hello".

Step 3: Create str2 = "HELLO".

Step 4: Compare both strings using
equalsIgnoreCase().

Step 5: Both strings contain the same letters,
ignoring uppercase and lowercase.

Step 6: Print true.

Step 7: Stop the program.

Algorithm:

START

Declare str1 = "Hello"

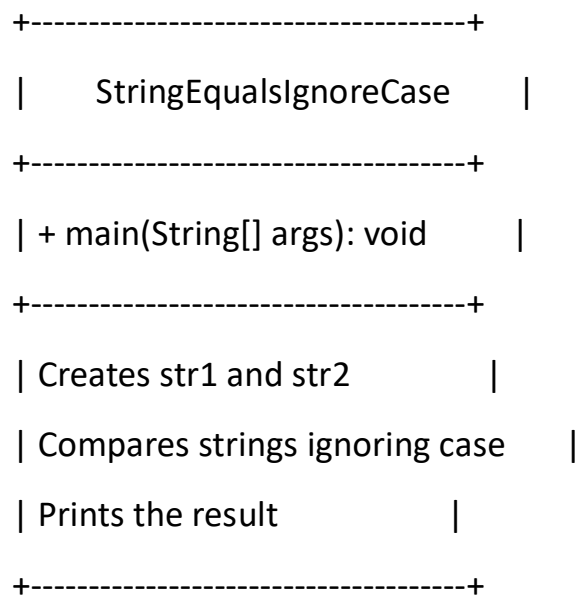
Declare str2 = "HELLO"

Compare using equalsIgnoreCase()

Print the result

STOP

UML Class Diagram:



Output:

true

8.StringIndexOf

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello".

Step 3: Find the position of character 'l'
using indexOf().

Step 4: The first 'l' is at index 2.

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

START

Declare str = "Hello

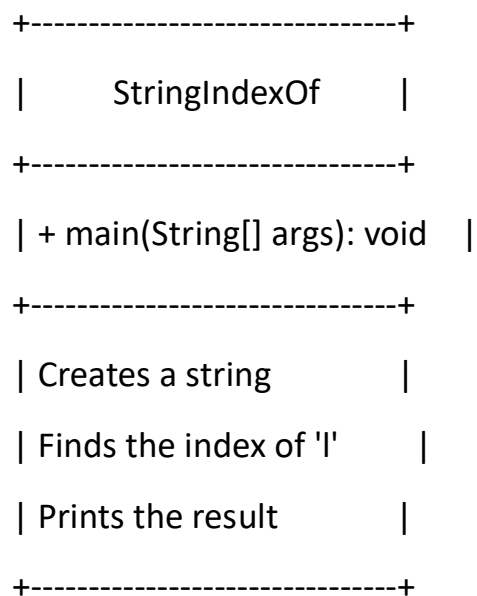
Find the first occurrence of 'l'

using indexOf()

Print the result

STOP

UML Class Diagram:



Output:

2

9.StringLastIndexOf

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello".

Step 3: Find the last occurrence of 'l'
using lastIndexOf().

Step 4: The last 'l' is at index 3.

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

STAR

Declare str = "Hello"

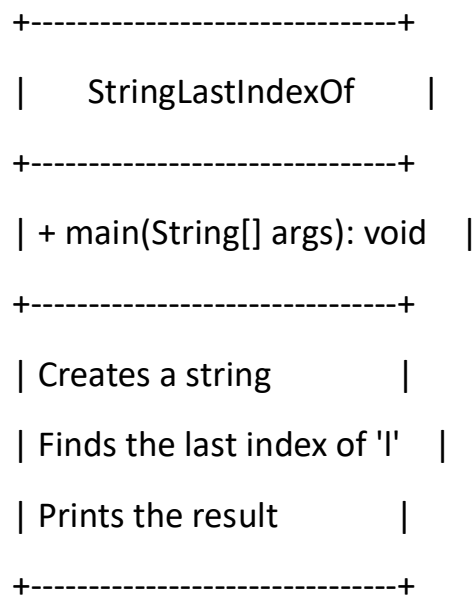
Find the last occurrence of 'l'

using lastIndexOf()

Print the result

STOP

UML Class Diagram:



Output:

3

10.String Length

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello".

Step 3: Find the number of characters
using length().

Step 4: The string contains 5 characters.

Step 5: Print the length.

Step 6: Stop the program.

Algorithm:

STAR

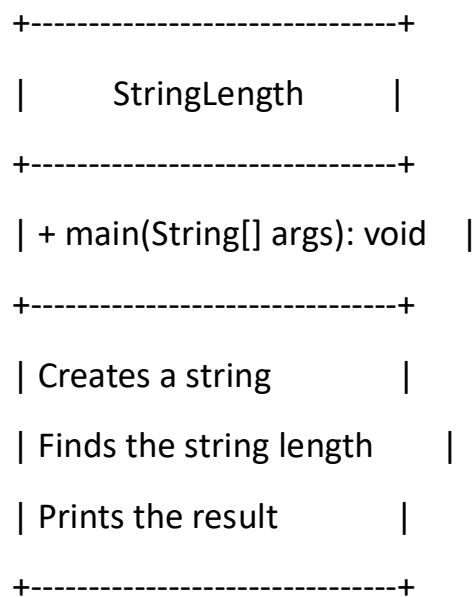
Declare str = "Hello"

Find the length using length()

Print the length

STOP

UML Class Diagram:



Output:

Length: 5

11.StringLower

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "HELLO".

Step 3: Convert the string into lowercase
using toLowerCase().

Step 4: The string becomes "hello".

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

STAR

Declare str = "HELLO"

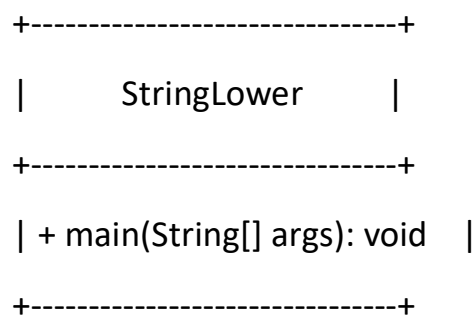
Convert str to lowercase

using toLowerCase()

Print the result

STOP

UML Class Diagram:



```
| Creates a string      |  
| Converts to lowercase |  
| Prints the result    |  
+-----+
```

Output:

hello

12.StringReplace

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello".

Step 3: Replace all occurrences of 'l'
with 'x' using replace().

Step 4: The string becomes "Hexxo".

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

STAR

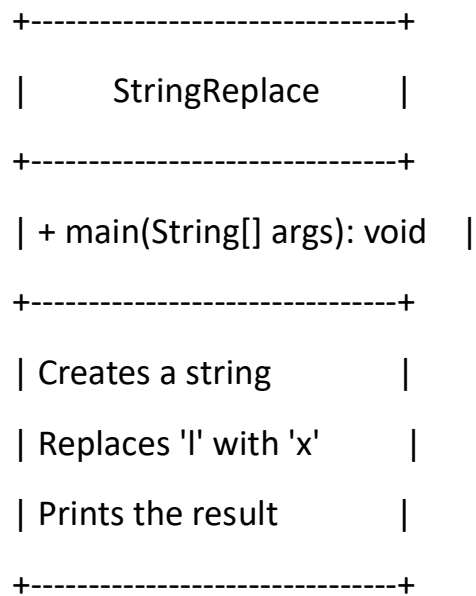
Declare str = "Hello"

Replace 'l' with 'x'

Print the result

STOP

UML Class Diagram:



Output:

Hexxo

13.StringStartsWith

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello Java".

Step 3: Check whether the string starts with
"Hello" using startsWith().

Step 4: The string starts with "Hello".

Step 5: Print true.

Step 6: Stop the program.

Algorithm:

STAR

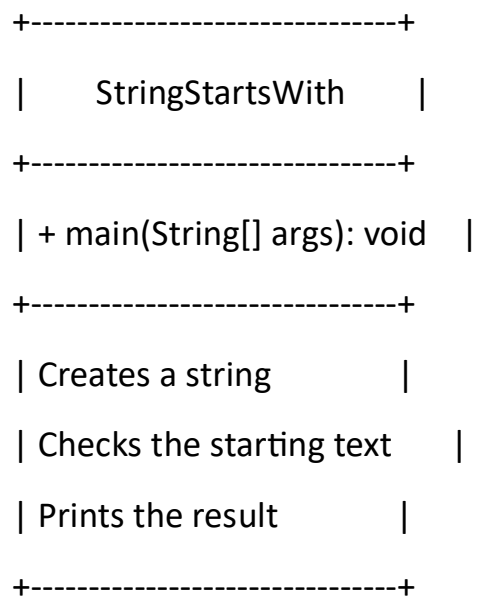
Declare str = "Hello Java"

Check startsWith("Hello")

Print the result

STOP

UML Class Diagram:



Output:

true

14.StringSubstring

Execution Steps:

Step 1: Start the program.

Step 2: Create a string str = "Hello Java".

Step 3: Extract the substring starting from
index 6 using substring(6).

Step 4: The substring is "Java".

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

START

Declare str = "Hello Java"

Extract substring from index

Print the result

STOP

UML Class Diagram:

+-----+

```

|   StringSubstring   |
+-----+
| + main(String[] args): void |
+-----+
| Creates a string      |
| Extracts a substring  |
| Prints the result     |
+-----+

```

Output:

Java

15.StringTrim

Execution Steps:

Step 1: Start the program.

Step 2: Create a string `str = " Hello Java "` with spaces at the beginning and end.

Step 3: Remove the leading and trailing spaces using `trim()`.

Step 4: The string becomes "Hello Java".

Step 5: Print the result.

Step 6: Stop the program.

Algorithm:

START

Declare str = " Hello Java "

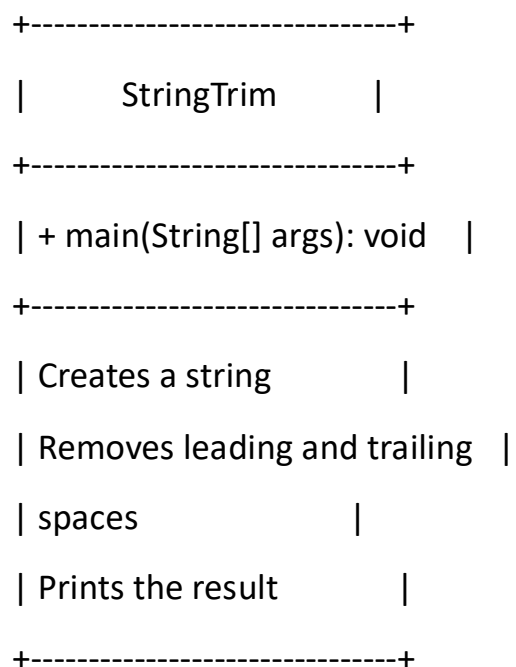
Remove leading and trailing spaces

using trim()

Print the result

STOP

UML Class Diagram:



Output:

Hello Java